Task 56: Rotting Oranges

Problem Description

You are given an m x n grid where each cell can have one of three values: 0 representing an empty cell, 1 representing a fresh orange, or 2 representing a rotten orange. Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Java Solution

```
class Solution {
    public int orangesRotting(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        Queue<int[]> queue = new LinkedList<>();
        int freshCount = 0;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 2) {
                    queue.offer(new int[]{i, j});
                } else if (grid[i][j] == 1) {
                    freshCount++;
                }
            }
        }

        if (freshCount == 0) return 0;

        int minutes = 0;
        int[][] dirs = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

        while (!queue.isEmpty() && freshCount > 0) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int[] curr = queue.poll();
                for (int[] dir : dirs) {
                    int r = curr[0] + dir[0];
                    int c = curr[1] + dir[1];
                    if (r >= 0 && r < m && c >= 0 && c < n && grid[r][c]
== 1) {
                        grid[r][c] = 2;
                        freshCount--;
                        queue.offer(new int[]{r, c});
                    }
                }
            }
            minutes++;
        }

        return freshCount == 0 ? minutes : -1;
    }
}
```